CHAPTER 14

Testing External Services

The one thing your project-management tool really needs is a bit of graphical spark. Specifically, you've been asked to have users' Twitter avatars show up on the site attached to tasks they have completed. (Handily, Twitter has a Ruby gem that's not too hard to set up.) Since this is *Rails 5 Test Prescriptions* and not *Rails 5 Connecting to Twitter Prescriptions*, you'd like to be able to test your interaction with the Twitter API.

Unfortunately, interacting with a third-party web service introduces a lot of complexity to your testing. Connecting to a web service is slow—even slower than the database connections you've already tried to avoid. Plus, connecting to a web service requires an Internet connection, and you'd like your test suite to be able to run on a train, on a boat, or during a network outage. Some external services are public—you don't want to post an update to Twitter every time you run tests, let alone post a credit-card payment to PayPal. Some services have rate limits, some cost money to access, some actually deal with money, and some require API keys and authentication.

The point is, you'd really, really like to be able to write and execute tests without hitting the service more than strictly necessary.

The strategies we've developed so far for test isolation and for using mocks to limit the scope of tests will help you successfully test an external service. You'll be able to test it on a train and test it on a boat and test it in the rain and test it on a goat.

External Testing Strategy

Your external-service testing story has two main characters:

- The *client*, the part of your application code that uses the external API, either because it needs data accessible via the API or because it's sending